

Live Cell Painting (LCP): pipeline completo, da aquisição ao Go/No-Go

Curso de HCI/HCA — tutorial prático, do cookiecutter ao pipeline de perfilamento fenotípico

NanoCell Interactions Lab

2026-08-03

Índice

0.1	Visão geral do fluxo	2
0.2	Criando o projeto com o cookiecutter-lcp	3
0.3	Criando um novo experimento	5
0.4	Preparando os metadados	7
0.5	Carregando os metadados no CellProfiler	10
0.6	AssayDev — controle de qualidade da segmentação	11
0.7	Analysis — extração de features em toda a placa	12
0.8	Do CellProfiler ao pipeline de análise	13
0.9	Ambiente Pixi e notebooks marimo	14
0.10	Percorrendo o pipeline: NB01 → NB07	14
0.11	Métricas de qualidade e a decisão Go/No-Go	16
0.12	Checklist final	17
0.13	Para saber mais	18

Este tutorial percorre o pipeline de Live Cell Painting (LCP) de ponta a ponta: criar o projeto, organizar metadados, rodar o CellProfiler (AssayDev → Analysis) e, a partir daí, rodar o pipeline de análise em Python/marimo até a decisão de Go/No-Go. A ideia é que você consiga acompanhar copiando e colando os comandos, na ordem em que aparecem.

i Antes de começar

- Se você ainda não sabe o que é e por que existe o Live Cell Painting, comece pela aula [Live Cell Painting — Histórico e Visão geral](#) do curso. Este tutorial é a parte prática — assume que a parte conceitual já foi vista.
- Você precisa ter CellProfiler com o plugin RunCellpose instalado — veja o tutorial [Instalar CellProfiler & Cellpose](#).
- Você precisa ter [Pypi](#) e [Git](#) instalados. Recomendamos também o [VS Code](#) — veja [Instalar VSCode](#) se ainda não usou.
- Recomendamos abrir este tutorial em uma aba do navegador e manter o terminal aberto em outra janela. No HTML, cada bloco de código tem um botão de cópia.

0.1 Visão geral do fluxo

O pipeline tem duas metades bem distintas, que conversam entre si através de uma pasta (workspace/backend/):

Aquisição (Cytation 5)

- Metadados (platemap + LoadData)
- AssayDev (QC de segmentação)
- Analysis (extração de features)
- backend/ (CSV / SQLite)
- NB01 → NB07 (pipeline marimo)
- Go / No-Go

Etapa	Ferramenta	O que produz
Aquisição	Cytation 5	imagens brutas por poço/site/canal
Metadados	Load Data Generator + Layout Generator	load_data.csv, platemap/, barcode_platemap.csv
AssayDev	CellProfiler (assaydev.cppipe)	segmentação testada em 1 site/poço, montagens de QC
Analysis	CellProfiler (analysis.cppipe)	features de todas as células, exportadas para backend/

Etapa	Ferramenta	O que produz
Pipeline de análise	Python + marimo (hca_pipeline)	perfis normalizados, PCA/UMAP/LDA, métricas de qualidade, decisão Go/No-Go

Cada etapa mora em uma pasta própria de workspace/, todas organizadas pelo mesmo identificador de experimento (EXPERIMENT_ID). Isso é o que garante que você sempre saiba onde procurar cada arquivo, independente de qual etapa do pipeline você está olhando.

0.2 Criando o projeto com o cookiecutter-lcp

Se você ainda não tem um projeto, instale o Cookiecutter (uma vez, no seu computador):

```
pip install --user pipx
pipx install cookiecutter
cookiecutter --version
```

Gere o projeto a partir do template do laboratório:

```
cookiecutter gh:labnanocell/cookiecutter-lcp
```

O Cookiecutter vai perguntar alguns campos. Os dois primeiros são obrigatórios; os demais são opcionais e ajudam a compor o nome do repositório:

```
author_name (Your Name): Marcelo Bispo
author_email (you@example.com): bispo@unicamp.br
org_name (nanocell-interactions-lab-unicamp):
modality (lcp):
project_tag (nanotoxicology):
cell_models (): Huh7
tissue ():
perturbation_category ():
nanoparticle (): npps
drug ():
genetic ():
```

O nome do repositório (repo_name) é montado automaticamente a partir desses campos — por exemplo lcp-nanotoxicology-huh7-npps. Ele não é o nome de um experimento específico: é o repositório “guarda-chuva” que vai acumular todos os experimentos do seu projeto de pós-graduação.

 Escolha o repo_name com cuidado

Trocar o nome do repositório depois é bem desestimulado (afeta o link do GitHub, do REDU, e qualquer coisa que você já tenha compartilhado). Pense nele como <modalidade>-<foco>-<modelo> e teste se descreve bem o projeto inteiro, não só o primeiro experimento.

Isso cria a estrutura abaixo (ainda sem nenhum experimento dentro):

```
<repo_name>/
├── images/
├── workspace/
│   ├── metadata/
│   │   └── templates/                # barcode_platemap.csv,
platemap_metadata.txt
│   ├── load_data_csv/
│   ├── pipelines/
│   │   └── templates/                # assaydev.cppipe, analysis.cppipe
│   ├── hca_pipeline/                # pacote Python compartilhado (config,
io, normalize, ...)
│   ├── assaydev/
│   ├── segmentation/
│   │   └── cellpose/
│   │       ├── model/                # modelos de segmentação já treinados
│   │       └── training/
│   ├── analysis/
│   │   └── templates/                # notebooks marimo 01→07
│   ├── profiles/
│   ├── backend/
│   └── models/
├── workspace_dl/
└── pixi.toml
```

Note que nenhuma pasta é por experimento ainda. As subpastas `templates/` são material permanente — cada vez que você começar um novo experimento, você vai copiar delas, não editá-las diretamente. `hca_pipeline/` e `segmentation/cellpose/{model,training}/` também não são por experimento: existem uma única vez e são compartilhados por todos os experimentos do repositório.

Feito isso, inicialize o Git e crie o repositório remoto:

```
cd <repo_name>
git status
git add .
git commit -m "Initial scaffold via cookiecutter-lcp"

git remote add origin <URL-do-seu-repo-no-GitHub>
git branch -M main
git push -u origin main
```

Instale o ambiente computacional (Pixi cuida de todas as dependências Python — `marimo`, `pandas`, `scikit-learn`, `pycytominer`, `copairs`, etc.):

```
pixi install
pixi run check
```

`pixi run check` roda `scripts/check_environment.py`, que apenas tenta importar cada pacote necessário. Se ele imprimir `Environment check passed.`, está tudo certo.

0.3 Criando um novo experimento

Cada experimento recebe um `EXPERIMENT_ID` no formato:

```
YYYY_MM_DD_CellLine_Treatment_Condition
```

Exemplo: `2025_06_28_Huh7_NPPS_24h`. Defina a variável de ambiente e crie a estrutura de pastas desse experimento, copiando o conteúdo de cada `templates/`:

```
EXP=2025_06_28_Huh7_NPPS_24h
```

```
# imagens e (opcional) correção de iluminação
```

```
mkdir -p images/$EXP/images
```

```
mkdir -p images/$EXP/illum
```

```
# metadados – copiado de templates/, você vai editar em seguida
```

```
mkdir -p workspace/metadata/$EXP/platemap
```

```
cp workspace/metadata/templates/* workspace/metadata/$EXP/
```

```
# pipelines .cppipe – copiado de templates/, você vai adaptar em seguida
```

```
mkdir -p workspace/pipelines/$EXP
```

```
cp workspace/pipelines/templates/* workspace/pipelines/$EXP/
```

```
# QC de segmentação e saída da segmentação real
```

```
mkdir -p workspace/assaydev/$EXP/outlines_qc
```

```
mkdir -p workspace/segmentation/cellpose/objects/$EXP
```

```
# notebooks de análise – copiado de templates/
```

```
mkdir -p workspace/analysis/$EXP/analysis
```

```
cp workspace/analysis/templates/* workspace/analysis/$EXP/analysis/
```

```
# saídas pesadas (não sobem para o GitHub) e perfis
```

```
mkdir -p workspace/backend/$EXP
```

```
mkdir -p workspace/profiles/$EXP
```

```
mkdir -p workspace/models/$EXP
```

```
mkdir -p workspace_dl/$EXP/notebooks
```

Confirme com `tree -L 4` (ou `find . -maxdepth 4`) que a estrutura ficou como esperado antes de seguir para a próxima etapa. A partir daqui, todo o resto do tutorial usa esse mesmo `EXPERIMENT_ID`.

 Depois de criado, evite renomear

Uma vez que você começou a preencher metadados e a rodar o CellProfiler apontando para esse `EXPERIMENT_ID`, renomeá-lo depois significa corrigir caminhos em vários lugares (`.cppipe`, `load_data.csv`, `barcode_platemap.csv`). Vale a pena checar o nome com calma antes de seguir.

0.4 Preparando os metadados

Esta etapa cria os arquivos que vão substituir os quatro primeiros módulos do CellProfiler (Images, Metadata, NamesAndTypes, Groups) por um único módulo LoadData — muito mais reprodutível e menos sujeito a erro humano.

0.4.1 1. Load Data Generator

O laboratório usa dois programas (atalhos nos computadores do laboratório) pensados para o Live Cell Painting no Cytation 5:

- Load Data Generator (LDG) — gera os arquivos com a localização e as informações mínimas de cada imagem;
- Layout Generator — converte o *platemap* em arquivos de metadados experimentais.

Rodando o LDG:

1. Selecione a pasta com as imagens do experimento, por exemplo:

```
images/$EXP/images/protocolo_aquisicao/
```

Não renomeie pastas ou arquivos depois deste passo — o CellProfiler perde a referência.

2. O programa reconhece os canais pelo nome dos arquivos. No Live Cell Painting, o padrão é:
 - GFP → AOGFP
 - Propidium Iodide → AOPI
 - DAPI (se usado) → Nuclei
3. Clique OK. Ele roda rápido e fecha, criando uma subpasta `load_data_csv/` dentro da pasta de imagens.

Se você tem replicatas biológicas feitas em dias diferentes, repita o processo uma vez por dia/-pasta.

A pasta `load_data_csv/` gerada contém três arquivos:

Arquivo	Conteúdo
<code>load_data.csv</code>	referência principal: canais, poços, sites, placas
<code>load_data_with_illum.csv</code>	só é usado se houver correção de iluminação (raro no Cytation 5)
<code>metadata_representative_cells.csv</code>	usado depois para identificar imagens representativas

Depois de confirmar que os três arquivos foram gerados corretamente, mova (ou renomeie por replicata, ex. `load_data_csv_R1/`) esse conteúdo para `workspace/load_data_csv/$EXP/` no seu projeto.

0.4.2 2. Platemap e Layout Generator

O Layout Generator precisa de um platemap: uma tabela em que cada poço interno contém exatamente 4 campos, separados por um único espaço, na ordem:

Linhagem Tratamento Concentração Tipo

Exemplo válido: `Huh7 NPPS 1000 trt.`

Campo	Descrição	Exemplo	Observação
1. Linhagem	linhagem celular	Huh7	mesmo formato sempre; evite HUH7/Huh-7
2. Tratamento	composto/nanopartícula	NPPS	sem espaços (NonTreated, não Non treated)
3. Concentração	valor numérico	1000	sem unidade — documente a unidade separadamente

Campo	Descrição	Exemplo	Observação
4. Tipo	papel do poço	trt, negcon, poscon	minúsculo, sem espaço

A primeira linha e a primeira coluna da tabela ficam vazias (representam as bordas da placa). Poços que foram usados só para ajustar a aquisição (“poços de sacrifício”) devem ficar em branco, não recebem as 4 informações — caso contrário a análise vai tentar referenciar imagens que não existem.

 O Layout Generator é sensível a inconsistências

HUH-7 Polystyrene_0.42um C-1-0.1g/L trt não vai ser reconhecido — o resultado sai sem os seus dados. Mantenha o padrão de 4 campos, um espaço, rigorosamente.

Passo a passo no Layout Generator:

1. Informe o número de poços da placa (ex.: 96).
2. Selecione o arquivo de platemap.
3. Preencha os rótulos de cada nome único encontrado (Cell_Type, Treatment, Concentration, Control_Type).
4. Escolha como pasta de saída a pasta metadata/ do seu projeto. O programa cria uma sub-pasta platemap/ com o layout convertido (layout_dayN.csv e .txt).

Se o experimento foi feito em vários dias independentes, repita para cada dia.

0.4.3 3. barcode_platemap.csv

Esse arquivo é feito manualmente e liga cada placa física (imageada em um dia específico) ao seu platemap:

Assay_Plate_Barcode	Plate_Map_Name
220505_095645_Plate_1	layout_day1.txt
220614_143057_Plate_1	layout_day2.txt

- Assay_Plate_Barcode = nome da pasta de imagens daquela réplica.
- Plate_Map_Name = nome do arquivo de layout gerado no passo anterior.

Salve como workspace/metadata/\$EXP/barcode_platemap.csv (o template já deixou um exem-

plo em `workspace/metadata/templates/`, adapte-o em vez de criar do zero).

Ao final desta etapa você deve ter, dentro de `workspace/metadata/$EXP/`:

```
workspace/metadata/2025_06_28_Huh7_NPPS_24h/  
├─ barcode_platemap.csv  
└─ platemap/  
    ├─ layout_day1.csv  
    └─ layout_day1.txt
```

0.5 Carregando os metadados no CellProfiler

Abra o CellProfiler no ambiente com os plugins (Cellpose incluído) — veja [Instalar CellProfiler & Cellpose](#) se ainda não configurou.

```
conda activate cp_cellpose  
cellprofiler
```

Aviso

No Windows, permaneça no drive C: para abrir o CellProfiler — abrir a partir de outro drive (ex. D:) costuma gerar erro.

Copie `assaydev.cppipe` e `analysis.cppipe` de `workspace/pipelines/$EXP/` (você já os copiou de `templates/` na etapa de criação do experimento) e abra o `assaydev.cppipe` primeiro:

1. Adicione o módulo LoadData (+ → buscar “LoadData”). O CellProfiler vai perguntar se você quer remover Images, Metadata, NamesAndTypes e Groups — confirme com OK.
2. Em Input data file location, indique a pasta onde está `load_data.csv` (não o arquivo em si).
3. Selecione o arquivo `load_data.csv`. Use o botão View para confirmar que canais, poços, sites e placas foram lidos corretamente.
4. Ajustes recomendados:
 - Load images based on this data? → Yes
 - Rescale intensities? → Yes
 - Base image location → confirme que aponta para a pasta real de imagens (ajuste se estiver em Default Input Folder)
 - Select metadata tags for grouping → normalmente Well, Site, Plate
 - Process just a range of rows? → No (só use Yes para testar em um subconjunto)

Teste a configuração adicionando um módulo `IdentifyPrimaryObjects` temporário, escolhendo `A0GFP` como imagem de entrada, e rodando `Start Test Mode`. Você deve ver uma tabela com as colunas `FileName_A0GFP`, `FileName_A0PI`, `Metadata_Well`, `Metadata_Site`, `Metadata_Plate` preenchidas corretamente.

0.6 AssayDev — controle de qualidade da segmentação

O AssayDev ajusta os parâmetros de segmentação em um único site por poço, antes de rodar o experimento inteiro. O objetivo é uma segmentação *razoável e consistente* — não perfeita. Erros ocasionais e isolados (ex.: uma “célula gigante” por baixo sinal citoplasmático) não são preocupantes e podem ser filtrados depois; erros sistemáticos, que se repetem com o mesmo padrão, são o que realmente merece atenção, porque introduzem viés.

Mantenha núcleos/células que tocam a borda da imagem

Excluir objetos na borda costuma fazer com que células vizinhas “adotem” esses núcleos como parte do próprio citoplasma, distorcendo o resultado. É mais seguro manter e filtrar depois com `FilterObjects`, se necessário.

No pipeline `assaydev.cppipe`, os módulos principais (já configurados no template, mas revise cada um) são:

Módulo	Função
<code>FlagImage</code>	seleciona 1 site por poço (metadado <code>Site</code> , faixa min/máx do site desejado)
<code>RunCellpose</code>	segmenta os núcleos a partir de <code>A0GFP</code> , usando um modelo pré-treinado → objeto <code>NucleiCP</code>
<code>FlagImage (2)</code>	descarta imagens com menos de 3 núcleos (campo pouco representativo)
<code>Smooth</code>	suaviza a imagem antes da segmentação de célula
<code>IdentifySecondaryObjects</code>	segmenta a célula completa a partir de <code>NucleiCP</code> → objeto <code>Cells</code>
<code>IdentifyTertiaryObjects</code>	$\text{citoplasma} = \text{Cells} - \text{NucleiCP}$ → objeto <code>Cytoplasm</code>
<code>RescaleIntensity + ImageMath</code>	equaliza intensidades de <code>GFP/PI</code> , facilita segmentar nucléolos/vesículas

Módulo	Função
IdentifyPrimaryObjects (nucléolos, vesículas)	nucléolos dentro do núcleo; vesículas no citoplasma, canal PI
OverlayOutlines + SaveImages	salva as imagens de QC (contornos + montagens)

Rodando:

1. Limpe as imagens que estavam carregadas no pipeline e adicione as do experimento atual.
2. Rode Update no módulo Metadata/LoadData e confirme que a extração fez sentido.
3. Rode Start Test Mode e revise os módulos um a um.
4. Rode o pipeline completo (assaydev.cppipe, ainda só 1 site/poço).
5. Avalie a pasta de saída — se a segmentação estiver *good enough*, mova as imagens de QC para workspace/assaydev/\$EXP/outlines_qc/.

! Checklist de AssayDev

- ☐ Pastas e metadados configurados
- ☐ LoadData funcionando (teste feito)
- ☐ FlagImage filtrando exatamente 1 site por poço
- ☐ Segmentação de núcleo/célula/citoplasma/nucléolo/vesícula ajustada
- ☐ QC via SaveImages funcionando (montagens + outlines)

0.7 Analysis — extração de features em toda a placa

Com a segmentação validada no AssayDev, é hora de rodar a análise completa. O objetivo não é um pipeline “perfeito” — um pipeline perfeito para uma imagem específica está provavelmente overfitado e vai falhar em outras.

1. Abra analysis.cppipe (o pipeline base, já com LoadData configurado) em uma segunda janela do CellProfiler, mantendo assaydev.cppipe aberto como referência.
2. Arraste os módulos de segmentação do assaydev.cppipe para o analysis.cppipe, logo depois do LoadData: RunCellpose, Smooth, IdentifySecondaryObjects, IdentifyTertiaryObjects, RescaleIntensity, ImageMath, EnhanceOrSuppressFeatures, MaskImage, IdentifyPrimaryObjects. Você não precisa do FlagImage de 1-site-por-poço — agora queremos todas as imagens.
3. Confirme que o LoadData do analysis.cppipe aponta para o mesmo load_data.csv deste

experimento.

4. Garanta que as medidas (ExportToSpreadsheet / módulos Measure*) estão sendo feitas sobre as imagens raw (A0GFP, A0PI), não sobre imagens intermediárias criadas só para ajudar a segmentação.

Rastreando o fluxo de um objeto

Clique com o botão direito em um módulo de análise e escolha Trace inputs/outputs. Isso mostra de onde vem cada dado e onde ele é usado — útil para confirmar que nada ficou desconectado ao copiar módulos entre pipelines.

Configure o CellProfiler para salvar a saída como um banco SQLite (ou CSVs, se preferir) dentro de `workspace/backend/$EXP/`. Ative em `File > Preferences` a opção “Save pipeline and/or file list in addition to project” = Pipeline, para não depender de salvar manualmente.

Ao terminar de rodar, você deve ter em `workspace/backend/$EXP/` um banco `.sqlite` (ou um conjunto de `.csv`) contendo, para cada célula segmentada, centenas de *features* de forma, intensidade e textura, organizadas por três *compartimentos*:

Objeto	Descrição
Cells	célula completa
Cytoplasm	região entre núcleo e contorno celular
Nuclei	núcleo

0.8 Do CellProfiler ao pipeline de análise

A partir de agora, tudo acontece em Python — os notebooks lêem diretamente esse banco/CSV de `workspace/backend/$EXP/`.

Internamente, o CellProfiler produz um arquivo por compartimento (`Cells.csv`, `Cytoplasm.csv`, `Nuclei.csv` — ou tabelas equivalentes dentro do `.sqlite`). O `01_samples_retrieval.py` (primeiro notebook do pipeline) faz automaticamente o que antes era manual:

- prefixar cada *feature* com o compartimento de origem (`Cells_AreaShape_Area`, `Nuclei_AreaShape_Area`, ...), para não confundir *features* com o mesmo nome vindas de objetos diferentes;
- juntar os três compartimentos em uma única tabela por célula;
- consolidar todas as placas/réplicas do experimento em um único arquivo: sin-

```
gle_cell_profiles.parquet.
```

Você não precisa escrever esse código — é exatamente o papel do NB01. A partir daqui o tutorial passa a rodar dentro do ambiente Pixi.

0.9 Ambiente Pixi e notebooks marimo

Os notebooks de análise não são Jupyter — são notebooks **marimo**: arquivos `.py` puros (`app = marimo.App()`), reativos, sem estado de célula escondido.

Editar um notebook (abre o editor reativo no navegador):

```
pixi run marimo edit workspace/analysis/$EXP/analysis/01_samples_retrieval.py
```

Os widgets no topo de cada notebook (seletor de experimento, limiares, checkboxes) são elementos `mo.ui` — mudar um deles re-executa automaticamente todas as células que dependem dele.

Rodar um notebook sem interface (determinístico, cada `mo.ui` cai no seu valor padrão — é o que se usa para “só rodar o pipeline inteiro” ou em CI):

```
pixi run python3 workspace/analysis/$EXP/analysis/01_samples_retrieval.py
```

i Um detalhe de nomenclatura do marimo

Uma variável cujo nome começa com `_` é privada à célula onde foi criada. Se você editar um notebook e precisar que uma variável seja lida por outra célula, o nome não pode começar com `_` — senão `marimo check --fix` a remove silenciosamente das exportações da célula.

0.10 Percorrendo o pipeline: NB01 → NB07

`01_samples_retrieval.py`

- Entrada: CSV/SQLite do CellProfiler
- Saída: `single_cell_profiles.parquet`
- O que faz: lê e consolida os perfis, checagens estruturais (SC-01→SC-05)

`02_aggregate_normalize_featureselect.py`

- Entrada: `single_cell_profiles.parquet`
- Saída: `per_well_features_selected.parquet`
- O que faz: anota com o platemap, agrega para o nível de poço, normaliza (`mad_robustize`), seleciona *features* (SC-06→SC-10)

`03_phenotypic_profiling.py`

- Entrada: `per_well_features_selected.parquet`
- Saída: figuras/modelos de PCA, UMAP, LDA
- O que faz: PCA, UMAP, LDA com CV leave-one-plate-out, Cohen's d, KMeans, comparação sem correção vs. Harmony (SC-11→SC-13)

`04_phenotypic_fingerprints.py`

- Entrada: `per_well_features_selected.parquet`
- Saída: *fingerprints* por taxonomia de feature
- O que faz: taxonomia de features, Cohen's d por condição, *heatmaps*/radares — eixo de dose é opcional

`05_quality_metrics.py`

- Entrada: `per_well_features_selected.parquet`
- Saída: métricas de qualidade + decisão
- O que faz: Percent Replicating, mAP, avaliação de *batch*, dose-resposta, decisão Go/No-Go (SC-18→SC-22)

`06_single_cell_analysis.py`

- Entrada: `single_cell_ready.parquet` (cache do NB02)
- Saída: SHAP, relatório de classificação, figuras
- O que faz: PCA/UMAP no nível de célula única, HDBSCAN+KMeans, LightGBM+SHAP, distância de Mahalanobis (SC-14→SC-17)

`07_recovery_axis_analysis.py` (opcional)

- Entrada: `per_well_features_selected.parquet`
- Saída: eixo de recuperação
- O que faz: eixo geométrico entre dois estados de referência — só se aplica a desenhos experimentais com essa estrutura; pula automaticamente se não configurado

Rode-os em ordem, sempre revisando os widgets no topo antes de seguir para o próximo:

```
pixi run python3 workspace/analysis/$EXP/analysis/01_samples_retrieval.py
pixi run python3
workspace/analysis/$EXP/analysis/02_aggregate_normalize_featuresselect.py
pixi run python3 workspace/analysis/$EXP/analysis/03_phenotypic_profiling.py
pixi run python3 workspace/analysis/$EXP/analysis/04_phenotypic_fingerprints.py
pixi run python3 workspace/analysis/$EXP/analysis/05_quality_metrics.py
pixi run python3 workspace/analysis/$EXP/analysis/06_single_cell_analysis.py
```

Todos os notebooks importam de um único pacote Python compartilhado, `hca_pipeline` (`workspace/hca_pipeline/`) — é ele quem sabe ler `experiment_config.json`, aplicar a normalização, calcular as métricas de qualidade, etc. Você não precisa editar esse pacote para rodar o pipeline em um experimento novo; ele é escrito para ser agnóstico ao experimento.

0.11 Métricas de qualidade e a decisão Go/No-Go

O NB05 é o portão de qualidade do pipeline. Ele resume três métricas clássicas de *image-based profiling*:

Métrica	Pergunta que responde	Faixa	Bom sinal	Referência
PR (<i>Percent Replicating</i>)	Que fração dos tratamentos replica acima do nulo?	0–1	> 0.25	Caicedo et al. 2020
PM (<i>Percent Matching</i>)	Que fração dos perfis casa com o mesmo tratamento?	0–1	> 0.50	Caicedo et al. 2020
mAP (<i>Mean Average Precision</i>)	Quão bem conseguimos recuperar réplicas a partir do <i>ranking</i> ?	0–1	> 0.50, significativo (FDR)	Kalinin et al. 2025

E aplica os critérios de decisão:

Checação	ID	Crítica?	Condição de aprovação
Validação de controles	SC-19	Sim	PR do negcon ≤ 0.10 e PR do poscon > 0.50
Fração de PR	PR-25%	Sim	$\geq 25\%$ dos tratamentos passam PR por placa
Efeito de <i>batch</i>	SC-20	Não	queda de PR entre placas $< 30\%$ (só quando relevante)
Dose-resposta	SC-21	Não	dose-resposta monotônica para tratamentos multi-dose (pulado se não houver eixo de dose)
Detectabilidade do fenótipo	SC-22	Sim	mAP do poscon > 0.5 , todos os tratamentos mostram fenótipo

NB06 lê o relatório de Go/No-Go do NB05 e avisa se o experimento recebeu No-Go — vale revisar esse aviso antes de interpretar qualquer resultado de célula única.

! Se o resultado for No-Go

Um No-Go não significa necessariamente um experimento perdido: normalmente aponta para algo específico — controles mal separados, efeito de placa dominando o sinal, ou concentração/tempo fora da faixa detectável. Volte ao relatório do NB05 antes de decidir se é preciso repetir o experimento ou apenas ajustar a análise.

0.12 Checklist final

- ☐ Projeto criado com `cookiecutter-lcp`, `pixi install + pixi run check` passando
- ☐ `EXPERIMENT_ID` definido e pastas criadas a partir de templates/
- ☐ Metadados prontos: `barcode_platemap.csv` + `platemap/` preenchidos
- ☐ AssayDev revisado e QC salvo em `workspace/assaydev/$EXP/outlines_qc/`
- ☐ Analysis rodado, dados em `workspace/backend/$EXP/`

- ☐ NB01→NB06 rodados em ordem, revisando os widgets de cada um
- ☐ Relatório de Go/No-Go do NB05 lido e entendido

0.13 Para saber mais

- Já tem um `per_well_features_selected.parquet` pronto (de outro pipeline) e quer pular direto para o NB03? Veja o tutorial de bônus *Iniciando a análise a partir de per_well_features_selected.parquet*.
- Curiosidade sobre extensões futuras do framework de mAP (diagnóstico de efeito de placa, de *batch*, de dose) e sobre modos de falha do SHAP em dados de imagem: são notas de pesquisa em desenvolvimento no laboratório, fora do escopo deste tutorial introdutório.
- Referências centrais: Caicedo et al. (2020), *Nature Methods*, doi:10.1038/s41592-020-0851-3; Kalinin et al. (2025), *Nature Communications*, doi:10.1038/s41467-025-60306-2; Cimini et al. (2023), *Nature Protocols*, doi:10.1038/s41596-023-00840-9.